

Vineyard Primary School — School Management System (SMS)

Supervisor Presentation Documentation

Built with modern full-stack web + mobile technologies. Manages all aspects of school administration from a single platform.

Overview

The Vineyard SMS is a complete school administration platform built for Vineyard Primary School (Nairobi, Kenya). It covers student records, fee collection, staff management, academic operations, and communication — accessible from a web browser or a mobile app for staff on the go.

System Architecture

```
vineyard-school/  
├── packages/  
│   ├── server/           # Hono API server (runs on Bun)  
│   │   ├── src/routes/   # All API route handlers  
│   │   └── src/db/       # Drizzle ORM schema + migrations  
│   ├── web/              # React web frontend (Vite)  
│   │   └── src/web/  
│   │       ├── pages/    # All page components  
│   │       ├── components/ # Shared UI components  
│   │       └── lib/      # API client, auth helpers  
│   └── mobile/           # React Native / Expo mobile app  
│       └── src/  
│           ├── screens/  # Mobile screens  
│           └── components/  
├── package.json          # Monorepo root (Bun workspaces)  
└── README.md
```

Tech Stack:

Layer	Technology
Runtime	Bun
API Framework	Hono (TypeScript)
Database	SQLite via Drizzle ORM
Authentication	better-auth (sessions + Bearer tokens)
Web Frontend	React 18 + Vite + Wouter (routing)
Mobile	React Native (Expo)
State / Data	TanStack Query
Styling	Inline CSS (design system via CSS variables)
Fonts	Dancing Script (headings) + Poppins (body)

Features

Student Management

- Register, edit, deactivate students
- Admission number auto-generation
- Parent/guardian contact details
- Class assignment
- Student profile with full history

Staff Management

- Staff records (teachers, admin, support)
- Role and department assignment
- Contact information

Class Management

- Create and manage classes/streams
- Assign class teachers
- View enrolled students per class

Fee Management

- Fee structures per class and term
- Record fee payments (cash, M-Pesa, bank)
- Receipt number generation
- Outstanding balances and defaulter tracking
- WhatsApp reminders for defaulters (one-click)
- KES currency throughout

Attendance

- Daily attendance marking (Present / Absent / Late)
- View attendance by class and date
- Dashboard attendance summary

Examinations & Results

- Exam creation per term
- Enter marks per student per subject
- Automatic grade and comment generation
- Results listing per exam

Report Cards

- Generate individual student report cards
- Per-subject marks, grades, teacher remarks
- Printable PDF-style layout

Timetable

- Weekly timetable per class
- Period, subject, teacher assignment

Payroll

- Staff salary records
- Allowances and deductions
- Monthly payroll processing

Communication

- Internal announcement/message board
- WhatsApp deep-link integration for parent contact

Transport

- Vehicle and route management
- Student transport assignment

Library

- Book catalogue management
- Issue and return tracking

Inventory

- School asset and supply tracking
- Stock levels

Dashboard

- Live KPIs: total students, staff, fees collected, outstanding, net balance
- Today's attendance summary
- Fee defaulters widget
- Recent student enrollments
- Recent payments

API Endpoints

Base URL: `/api`

Authentication

Method	Endpoint	Description
POST	<code>/api/auth/sign-in/email</code>	Sign in with email + password
POST	<code>/api/auth/sign-up/email</code>	Create admin account
POST	<code>/api/auth/sign-out</code>	Sign out
GET	<code>/api/auth/session</code>	Get current session

Students

Method	Endpoint	Description
GET	/api/students	List all students
POST	/api/students	Add student
GET	/api/students/:id	Get student detail
PUT	/api/students/:id	Update student
DELETE	/api/students/:id	Delete student

Staff

Method	Endpoint	Description
GET	/api/staff	List all staff
POST	/api/staff	Add staff member
PUT	/api/staff/:id	Update staff
DELETE	/api/staff/:id	Delete staff

Classes

Method	Endpoint	Description
GET	/api/classes	List classes
POST	/api/classes	Create class
PUT	/api/classes/:id	Update class
DELETE	/api/classes/:id	Delete class

Fee Payments

Method	Endpoint	Description
GET	/api/fee-payments	All payments
POST	/api/fee-payments	Record payment
GET	/api/fee-payments/defaulters	Students with outstanding fees
GET	/api/fee-payments/student/:id	Payments for a student

Fee Structures

Method	Endpoint	Description
GET	/api/fee-structures	List fee structures
POST	/api/fee-structures	Create structure
PUT	/api/fee-structures/:id	Update structure

Attendance

Method	Endpoint	Description
GET	/api/attendance	List attendance records
POST	/api/attendance	Mark attendance
GET	/api/attendance/today	Today's summary

Exams & Results

Method	Endpoint	Description
GET	/api/exams	List exams
POST	/api/exams	Create exam
GET	/api/exam-results	List results
POST	/api/exam-results	Enter result

Dashboard

Method	Endpoint	Description
GET	<code>/api/dashboard/stats</code>	Aggregated KPI stats

Other Modules

- `/api/timetable` — Timetable entries
- `/api/payroll` — Payroll records
- `/api/report-cards` — Report card generation
- `/api/communication` — Messages/announcements
- `/api/transport` — Routes and vehicles
- `/api/library` — Book management
- `/api/inventory` — Asset/stock management

Authentication

The system uses **better-auth** for session management.

- **Web:** Cookie-based sessions (auto-handled by browser)
- **Mobile:** Bearer token authentication
- Sign in via `/api/auth/sign-in/email`
- Token returned in response header
- All subsequent requests include `Authorization: Bearer <token>`

Database

SQLite database managed by **Drizzle ORM**.

Key tables:

- `user` , `session` , `account` — Auth tables (better-auth managed)
- `students` — Student records
- `staff` — Staff records
- `classes` — Class/stream definitions
- `fee_structures` — Fee requirements per class/term
- `fee_payments` — Payment transactions

- `attendance` — Daily attendance records
 - `exams` — Exam definitions
 - `exam_results` — Student exam marks
 - `timetable_entries` — Weekly timetable slots
 - `payroll_records` — Staff payroll
 - `transport_routes` , `transport_vehicles` — Transport
 - `library_books` , `library_issues` — Library
 - `inventory_items` — Inventory
 - `communication_messages` — Announcements
-

How to Run

Prerequisites

- [Bun](#) installed
- Node.js 18+ (for tooling)

Setup

```
# Clone the repo
git clone https://github.com/nuru999/VINEYARD-SMS.git
cd VINEYARD-SMS

# Install dependencies
bun install

# Run database migrations
cd packages/server
bun run db:migrate
```

Development

```
# From repo root – starts both API server and web frontend
bun run dev

# Or individually:
# API server (port 3000)
cd packages/server && bun run dev

# Web frontend (port 4200)
cd packages/web && bunx vite

# Mobile (Expo)
cd packages/mobile && bun expo start
```

Production Build

```
# Build web frontend
cd packages/web && bunx vite build

# The server serves the built web files statically
cd packages/server && bun run start
```

Mobile App

The Expo mobile app provides full access to the SMS on the go:

- **Home** — Dashboard stats + quick actions
- **Students** — Search, view, add students
- **Attendance** — Mark and review attendance
- **Fees** — View payments, defaulters, record payments
- **Staff** — Staff directory and details

Authentication uses Bearer tokens. Build and distribute via Expo's EAS Build service.

Security

- All API routes protected by session/Bearer token middleware

- Passwords hashed with bcrypt via better-auth
 - CORS configured for trusted origins only
 - Input validation on all POST/PUT endpoints
-

Developed By

Built for Vineyard Primary School, Nairobi, Kenya.

School motto: *"Fruitful Development"*

© 2025 Vineyard Primary School. All rights reserved.